

Architecture logicielle

ERP Distribution, Vente & Livraison Laravel MySQL Flutter

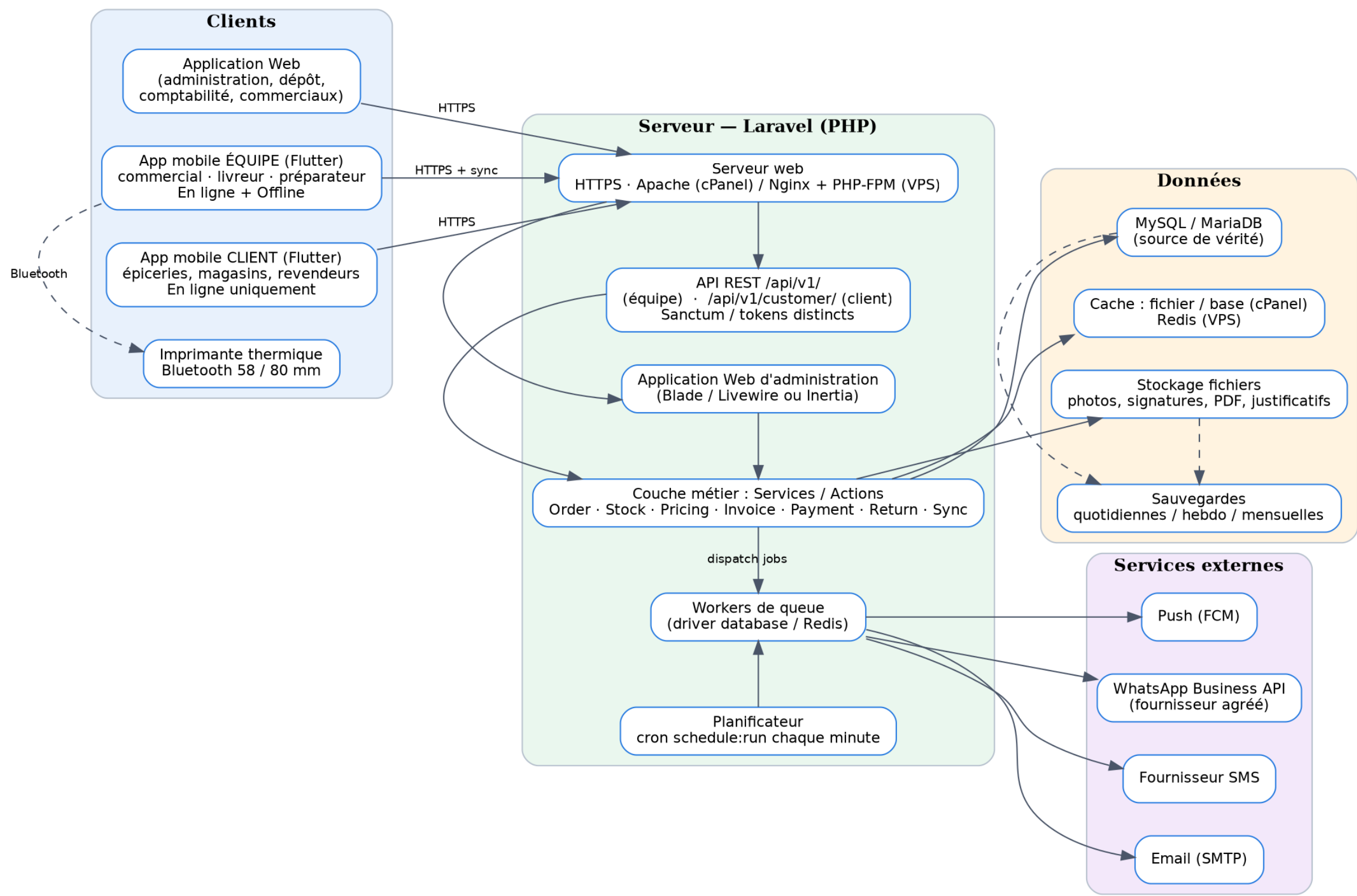
Contenu du document

- 1. Vue d'ensemble de la plateforme
- 2. Architecture en couches du backend Laravel
- 3. Découpage en modules métier
- 4. Arborescence du backend Laravel
- 5. Architecture de l'application mobile Flutter + arborescence
- 6. Synchronisation Offline
- 7. Sécurité et authentification
- 8. API REST (serveur et client)
- 9. Travaux de fond (queues, planificateur)
- 10. Hébergement cPanel / VPS
- 11. Environnements, Git et sauvegardes
- 12. Choix techniques proposés

Basé sur le cahier des charges ERP Distribution, Vente & Livraison v1.0 (28/09/2026). Les éléments marqués proposition sont des choix de conception ajoutés, absents du cahier des charges.

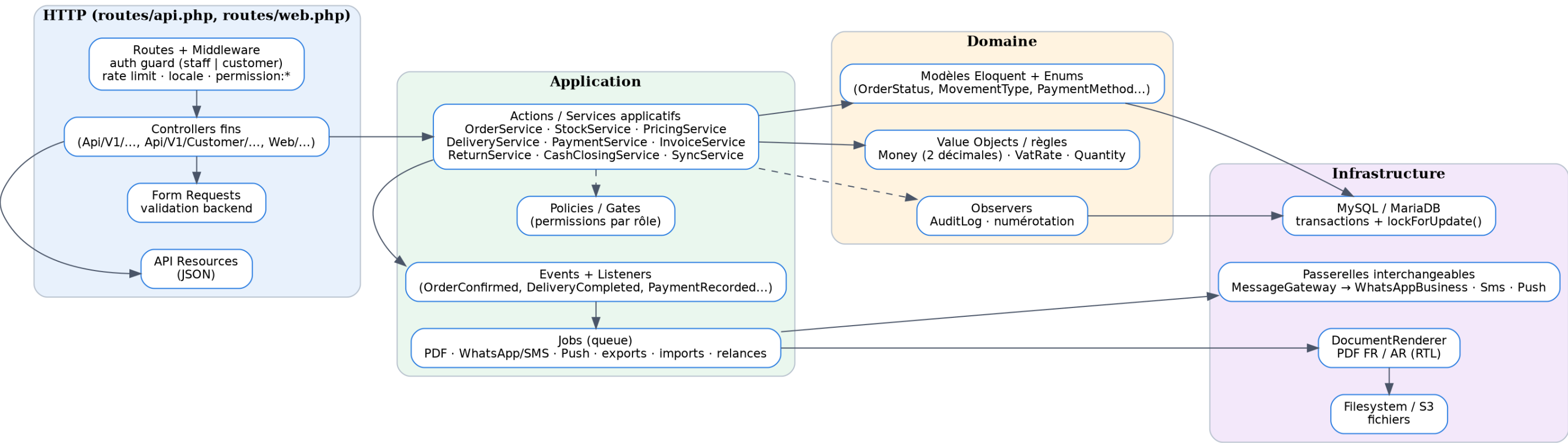
1. Vue d'ensemble de la plateforme

Trois clients (Web, mobile quipe, mobile client) une seule API Laravel MySQL ; les t ches lentes passent par la queue.



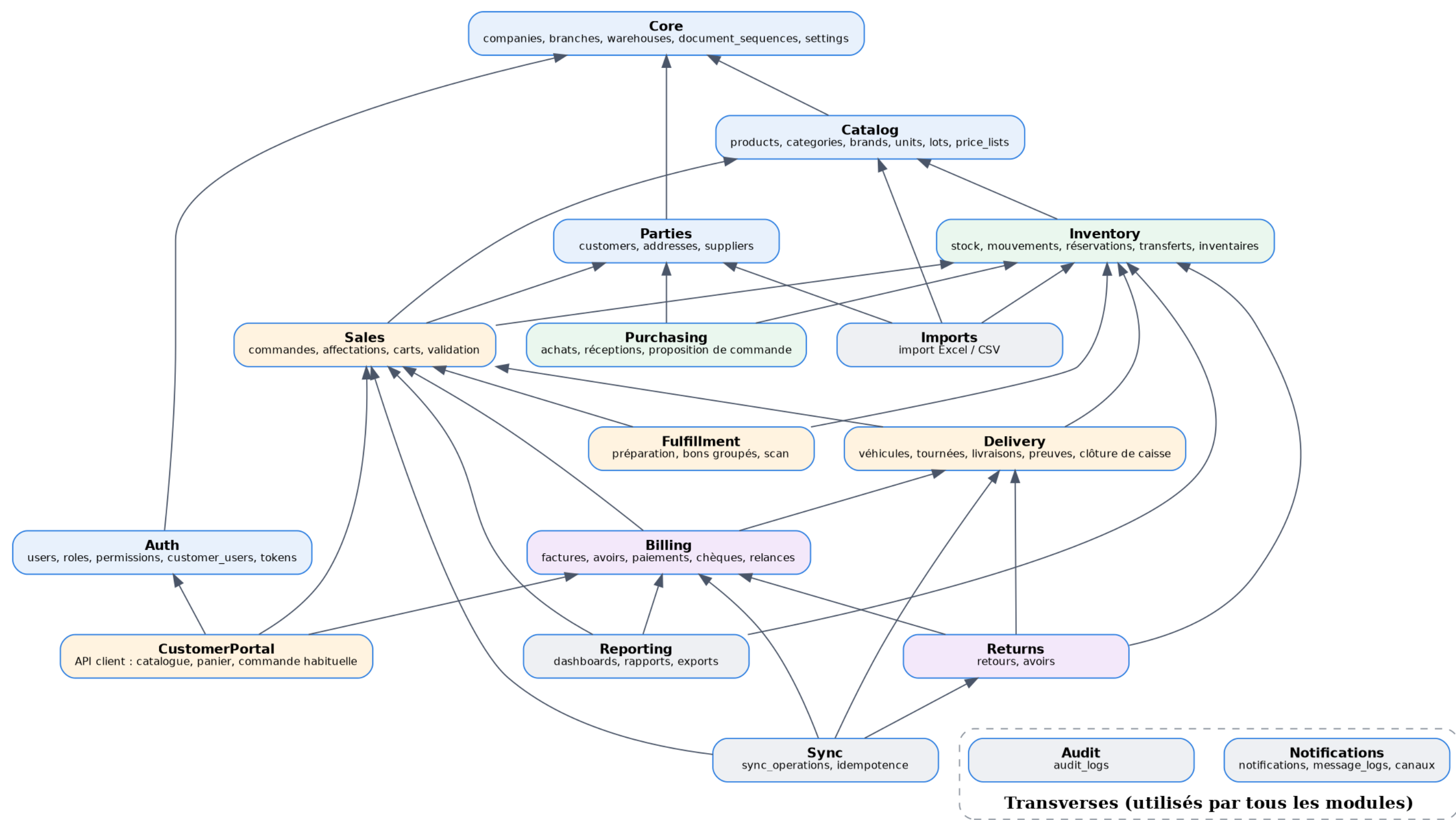
2. Architecture en couches du backend

Contrôleurs fins, logique dans les Services/Actions, calculs et stock c t serveur uniquement (source de v rit).



3. D coupage en modules m tier

Fl che = d pend de . Aucun module ne d pend d'un module situ au-dessus de lui (pas de cycle).

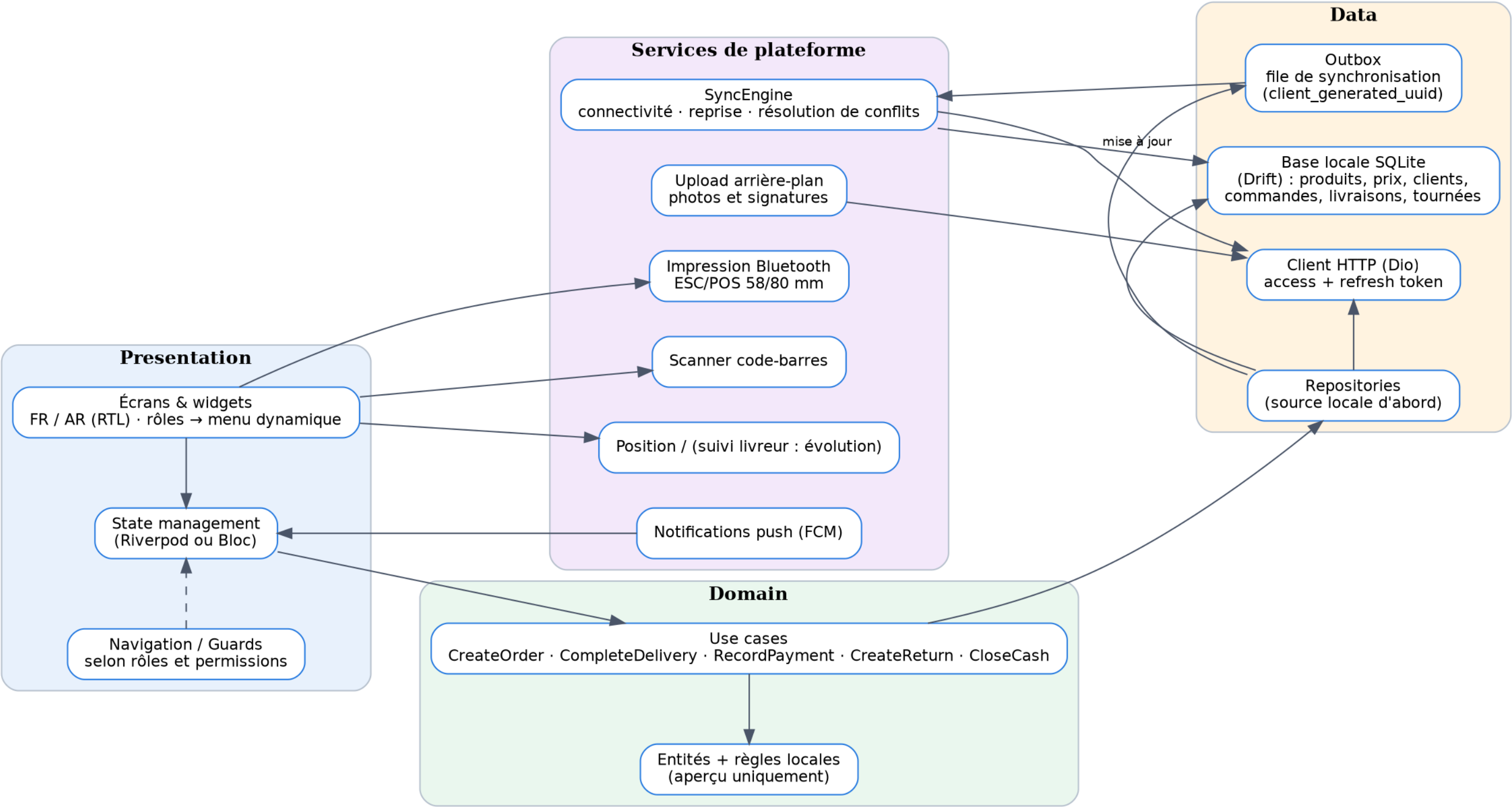


4. Arborescence du backend Laravel

```
erp-backend/          (Laravel  API REST versionn e + Web)
  app/
    Domain/           (un dossier par module m tier)
      Core/           Models, Services, Enums
      Auth/           User, Role, Permission, CustomerUser
      Catalog/        Product, Category, PriceList, ProductBatch
      Parties/        Customer, Supplier, CustomerAddress
      Inventory/       Stock, StockMovement, StockReservation, Transfer, Inventory
      Purchasing/      Purchase, ReorderSuggestion
      Sales/           Order, OrderLine, OrderAssignment, Cart
      Fulfillment/     PickingList, PickingListLine
      Delivery/        Route, Delivery, DeliveryProof, CashClosing, Vehicle
      Billing/          Invoice, Payment, Cheque, Reminder
      Returns/         ReturnOrder, ReturnLine
      Sync/            SyncOperation, SyncService
      Audit/           AuditLog, AuditService
    Http/
      Controllers/Api/V1/      ( quipe)
      Controllers/Api/V1/Customer/  (clients)
      Controllers/Web/          (administration)
      Middleware/ (SetLocale, EnsureCustomerScope, RateLimit )
      Requests/ (Form Requests)
      Resources/ (API Resources)
    Jobs/ SendWhatsAppDocument, GenerateInvoicePdf, SendReminders, ProcessImport
    Listeners/ Events/ Observers/ Policies/
    Infrastructure/
      Messaging/ MessageGateway, WhatsAppBusinessGateway, SmsGateway, PushGateway
      Documents/ DocumentRenderer (PDF FR / AR)
    Support/ Money, VatCalculator, DocumentNumber
  config/ (auth.php : guards staff + customer ; queue.php ; cache.php)
  database/ migrations/ seeders/ factories/
  routes/ api.php (v1) customer.php web.php console.php (schedule)
  resources/ views/ (Blade) pdf/ (mod les FR/AR) lang/ (fr, ar)
  storage/app/ (photos, signatures, PDF, imports)
  tests/ Unit/ Feature/ Api/ (isolation des donn es entre clients)
  .env.example (QUEUE_CONNECTION, CACHE_DRIVER, WHATSAPP_PROVIDER )
```

5. Application mobile Flutter architecture

Source locale d'abord (SQLite). Les opérations passent par l'outbox ; le serveur reste la source de vérité.



5 (suite). Arborescence Flutter

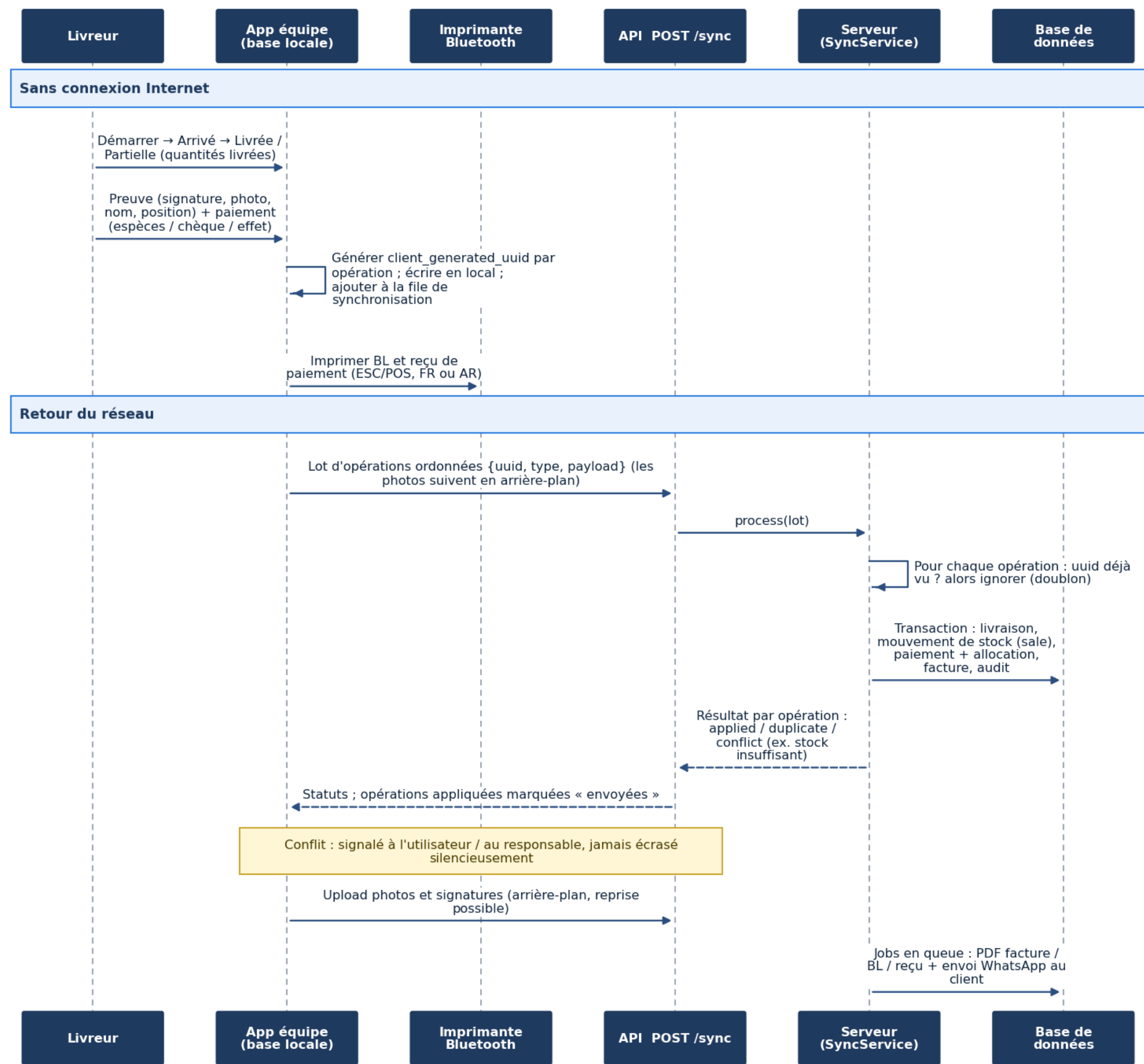
```
erp_mobile/                                (Flutter   une base de code, deux parcours : team / customer)
  lib/
    main_team.dart / main_customer.dart    (deux   flavors   )
    core/
      network/      dio_client, interceptors (token, refresh, locale)
      database/     drift (SQLite) : tables, DAO, migrations
      sync/         outbox, sync_engine, conflict_resolver, uploader
      l10n/         app_fr.arb, app_ar.arb (RTL)
      printing/     bluetooth_printer, escpos_templates (BL, re u, facture)
      services/     scanner, location, push, connectivity
    features_team/
      auth/  customers/  orders/  validation_queue/  preparation/
      delivery/ (tourn es, preuve, paiement)  returns/  cash_closing/  inventory/
      dashboard/
    features_customer/
      catalog/  cart/ (commande habituelle)  orders/ (recommander)
      invoices_balance/  profile/
    shared/      widgets, theme, formatters (MAD, dates)
  test/  integration_test/  (Offline, sync sans doublon, parcours complet)
  pubspec.yaml
```

6. Synchronisation Offline

Chaque opération créée hors connexion reçoit un **client_generated_uuid**. À la reconnexion, la file est envoyée dans l'ordre vers **POST /sync** ; le serveur ignore tout uuid déjà traité (idempotence), arbitre les conflits (stock insuffisant, prix modifié, client bloqué) et renvoie un résultat par opération : **applied**, **duplicate** ou **conflict**. Le téléphone n'est jamais la source définitive du stock, des prix, des factures, des paiements ni des soldes.

Cas	Comportement
Même opération envoyée deux fois	Une seule écriture (UNIQUE client_generated_uuid) ; réponse duplicate.
Stock insuffisant à la synchro d'une commande Offline	Opération conflict ; la commande reste à traiter, signalée au commercial / responsable.
Prix modifié côté serveur	Prix recalculés et figés par le serveur ; carte signalée.
Photos / signatures	Envoyées en arrière-plan, reprise automatique, rattachées par uuid.
Perte de réseau pendant l'envoi	Le lot est rejoué ; l'idempotence évite tout doublon.

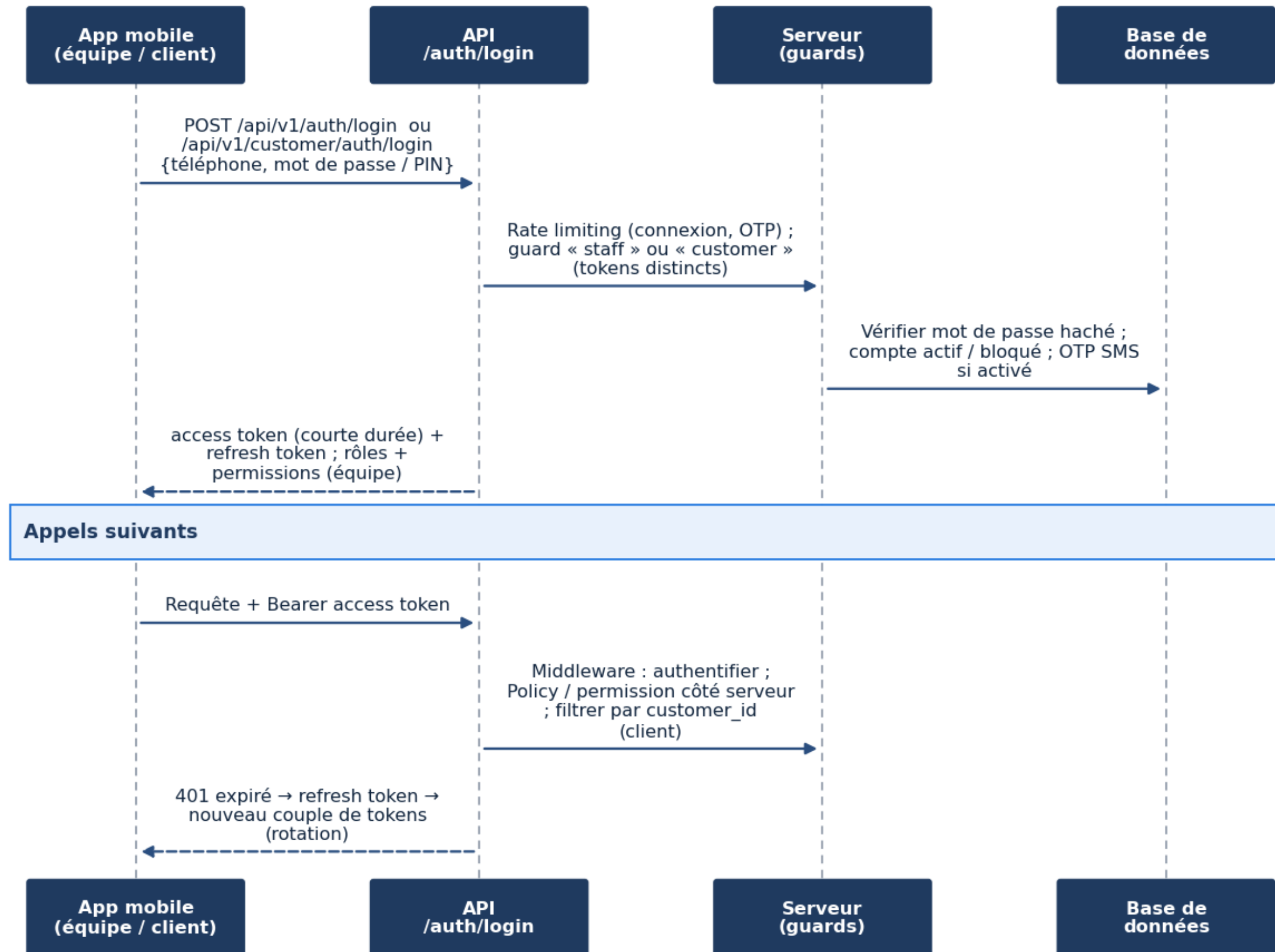
6 (suite). S quence de synchronisation Offline



7. Sécurité et authentification

Mesure	Mise en œuvre
Transport	HTTPS partout ; SSL AutoSSL (cPanel) ou Let's Encrypt (VPS).
Authentification	Deux guards séparés (staff, customer) ; access token + refresh token ; téléphone + mot de passe / PIN ; OTP SMS activable.
Autorisation	Permissions granulaires par rôle (ex. orders.validate) vérifiées côté serveur par Policies / Gates.
Isolation des clients	Global scope sur customer_id issu du token ; tests API dédiés à l'isolation des données.
Protection applicative	Rate limiting (connexion, OTP), validation backend (Form Requests), protection SQL injection / XSS / CSRF, chiffrement des données sensibles.
Traçabilité	audit_logs : auteur (utilisateur ou client), action, module, anciennes / nouvelles valeurs, IP, appareil.

7 (suite). S quence d'authentification



8. API REST versionné

Deux espaces d'API : **/api/v1/** (quipe) et **/api/v1/customer/** (clients), avec tokens distincts. Pagination, filtres et ressources JSON homog nes.

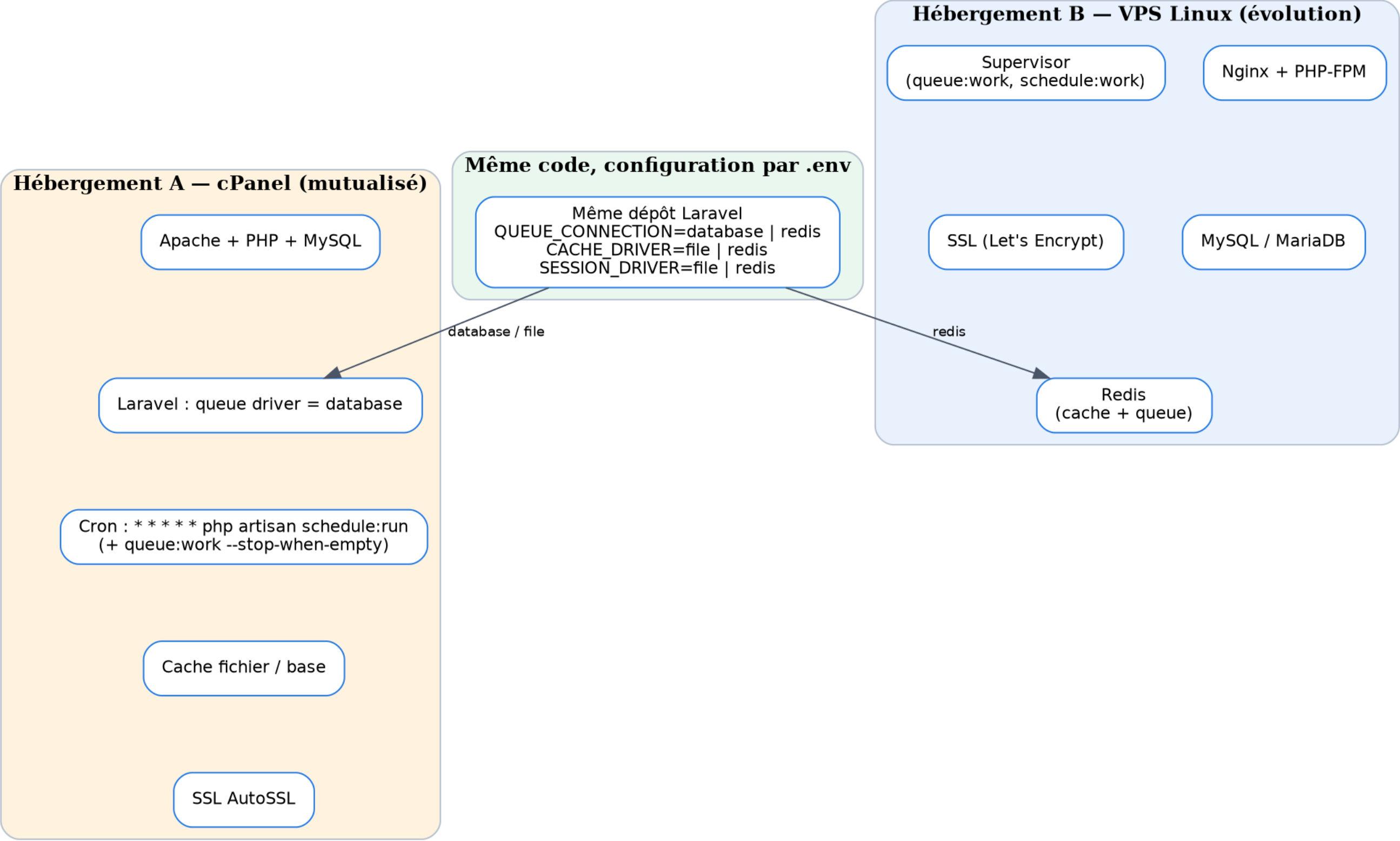
Groupe	quipe /api/v1/	Client /api/v1/customer/
Authentification	POST /auth/login	POST /auth/login
Catalogue	GET /products	GET /catalog GET /usual-order
Clients	GET /customers	GET /profile
Commandes	POST /orders GET /orders/{id} POST /orders/{id}/validate /reject /confirm /assign /prepare	GET /cart POST /cart/items POST /orders GET /orders POST /orders/{id}/cancel POST /orders/{id}/reorder
Pr paration	GET /picking-lists/{route}	
Livraison	POST /deliveries/{id}/start POST /deliveries/{id}/complete	
Finance	POST /payments POST /cash-closings POST /returns	GET /invoices GET /invoices/{id}/pdf GET /balance
Synchronisation Offline	POST /sync (lot d'op rations idempotentes)	

9. T ches de fond (queue Laravel) et planificateur

T che	D clencheur	Fr quence
G n ration des PDF (facture, BL, re u) FR / AR	Livraison, validation, demande manuelle	v nement
Envoi WhatsApp / SMS / email / push	v nements m tier (commande valid e, livr e)	v nement
Relances automatiques (J+1, J+7, J+15)	Facture chue (d lais configurables)	Planifi e quotidienne
Alerte clients inactifs	Aucune commande depuis X jours	Planifi e quotidienne
Alertes d' ch ance : ch ques, effets, lots, assurance / visite technique	Date d' ch ance proche	Planifi e quotidienne
Proposition de commande fournisseur	Stock seuil minimum	Planifi e quotidienne
Rappel jour de commande habituel (client)	Jour de commande du client	Planifi e quotidienne
Imports Excel / CSV et exports	Demande utilisateur	la demande
Sauvegardes (base, documents, photos, signatures)	Planificateur syst me	Quotidienne / hebdo / mensuelle

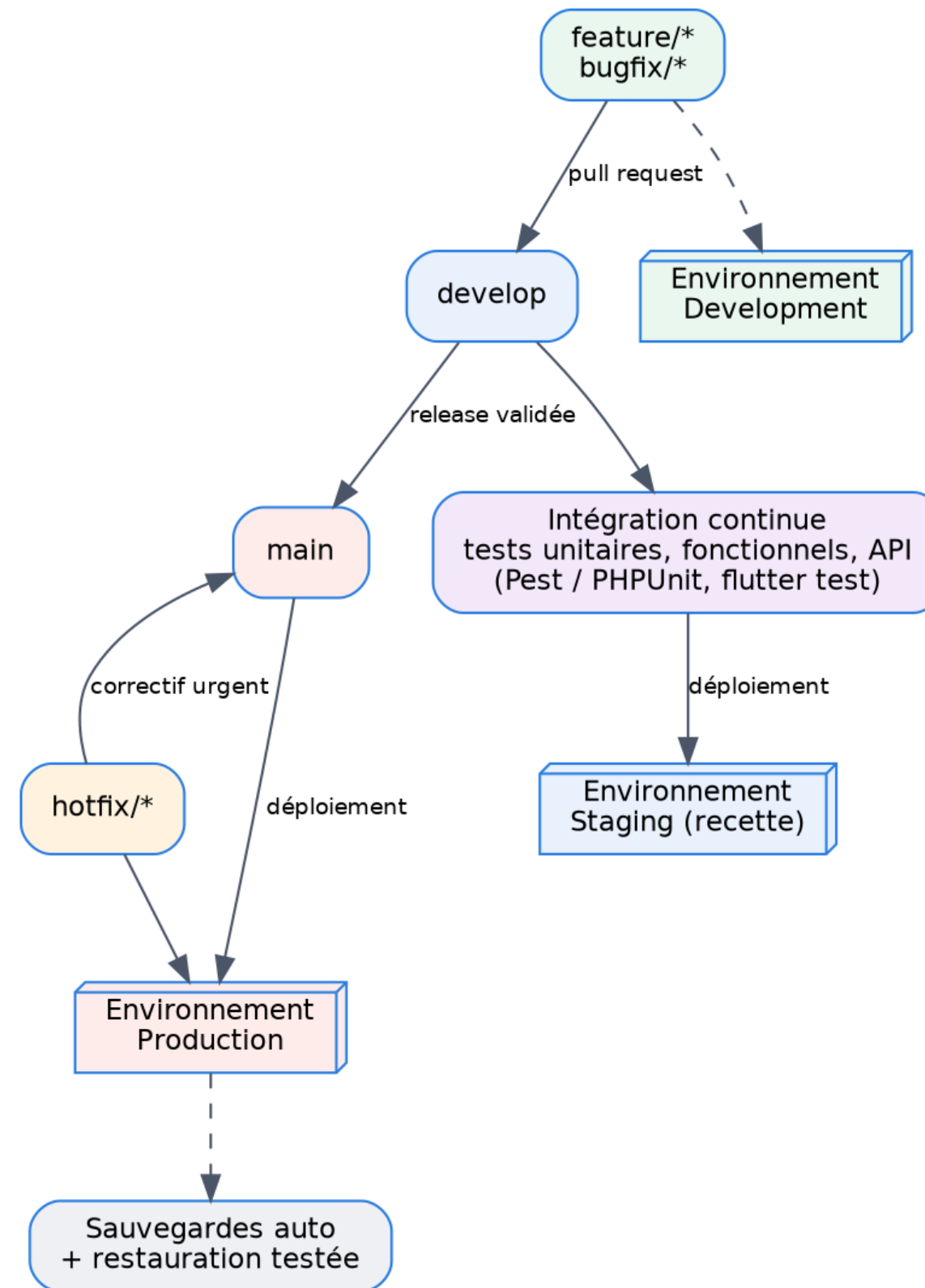
10. Hébergement : même code sur cPanel et sur VPS

Seule la configuration (.env) change : queue en base + cron sur cPanel ; Redis + Supervisor sur VPS.



11. Environnements, Git et sauvegardes

Development, Staging, Production ; aucun développement en production. Branches : main, develop, feature/*, bugfix/*, hotfix/.*



12. Choix techniques proposés

Le cahier des charges impose Laravel, MySQL/MariaDB et Flutter ; le reste est au choix du développeur. Les choix ci-dessous sont des **propositions**.

Besoin	Proposition	Pourquoi
Authentification API	Laravel Sanctum (+ refresh token applicatif)	Léger, fonctionne sur cPanel, abilities par guard.
Rôles et permissions	spatie/laravel-permission	Rôles multiples par utilisateur, permissions granulaires.
Application Web	Blade + Livewire (ou Inertia + Vue)	Productif pour un back-office ; déployable sur cPanel.
PDF FR / AR (RTL)	mPDF (ou dompdf avec police arabe)	Bon support de l'arabe et du RTL.
Import / export Excel	maatwebsite/excel	Modèles très facilement chargeables, rapport d'erreurs par ligne.
Queue	database (cPanel) Redis (VPS)	Imposé par le cahier des charges ; même code.
WhatsApp	Interface MessageGateway + fournisseur agréé	Service d'envoi interchangeable.
Flutter natif	Riverpod (ou Bloc)	Testable, adapté à l'offline-first.
Flutter base locale	Drift (SQLite)	Requêtes typées, migrations, transactions.
Flutter matériel	mobile_scanner, esc_pos + imprimante Bluetooth, signature_pad, geolocator, firebase_messaging	Scan code-barres, BL / reçu 58 / 80 mm, preuve de livraison, push.
Tests	Pest / PHPUnit ; flutter_test + integration_test	Couvre stock, TVA, retours, caisse, Offline, isolation clients.